

Tuần 29 - Thứ 2 buổi chiều

Matrix calculation, Array manipulation, Image as arrays intro

Thời gian: 14:00-17:00

Mục tiêu bài học

- Thực hiện các phép toán ma trận nâng cao
- Thao tác và xử lý arrays phức tạp
- Hiểu cách biểu diễn ảnh dưới dạng arrays
- Áp dụng kiến thức vào xử lý dữ liệu thực tế

Ôn tập buổi sáng

- Numpy arrays cơ bản
- Các phép toán đơn giản
- Shape, reshape, transpose

1. Matrix Calculation - Tính toán Ma trận

1.1 Ôn tập ma trận cơ bản

```
In [ ]: import numpy as np

# Tạo các ma trận để demo
print("=== Ôn tập ma trận cơ bản ===")

# Ma trận A (2x3)
A = np.array([[1, 2, 3],
              [4, 5, 6]])

# Ma trận B (2x3)
B = np.array([[7, 8, 9],
              [10, 11, 12]])

# Ma trận C (3x2) - để nhân được với A
C = np.array([[1, 2],
              [3, 4],
              [5, 6]])

print(f"Ma trận A (2x3):\n{A}")
```

```

print(f"\nMa trận B (2x3):\n{B}")
print(f"\nMa trận C (3x2):\n{C}")

# Các phép toán đã học buổi sáng
print("\n=== Phép toán element-wise (từng phần tử) ===")
print(f"A + B:\n{A + B}")
print(f"A * B (nhân từng phần tử):\n{A * B}")
print(f"A * 2:\n{A * 2}")

```

1.2 Phép nhân ma trận (Matrix Multiplication)

```

In [ ]: print("=== Phép nhân ma trận thực sự ===")

# Nhân ma trận A (2x3) với C (3x2) = kết quả (2x2)
# Điều kiện: số cột của A = số hàng của C
result_AC = np.dot(A, C) # hoặc A @ C
print(f"A (2x3) @ C (3x2) = Kết quả (2x2):")
print(f"A:\n{A}")
print(f"C:\n{C}")
print(f"A @ C:\n{result_AC}")

# Giải thích cách tính
print("\n=== Giải thích cách tính ===")
print("Phần tử [0,0] = 1*1 + 2*3 + 3*5 = 1 + 6 + 15 = 22")
print("Phần tử [0,1] = 1*2 + 2*4 + 3*6 = 2 + 8 + 18 = 28")
print("Phần tử [1,0] = 4*1 + 5*3 + 6*5 = 4 + 15 + 30 = 49")
print("Phần tử [1,1] = 4*2 + 5*4 + 6*6 = 8 + 20 + 36 = 64")

# So sánh với phép nhân element-wise
print("\n=== So sánh với element-wise multiplication ===")
print(f"A * B (element-wise, cùng kích thước):\n{A * B}")
print(f"A @ C (matrix multiplication, khác kích thước):\n{result_AC}")

```

1.3 Ma trận vuông và các phép toán đặc biệt

```

In [ ]: # Tạo ma trận vuông 3x3
square_matrix = np.array([[2, 1, 3],
                           [1, 4, 2],
                           [3, 2, 1]])

print(f"Ma trận vuông 3x3:\n{square_matrix}")

# Ma trận đơn vị (Identity matrix)
identity = np.eye(3) # Ma trận đơn vị 3x3
print(f"\nMa trận đơn vị 3x3:\n{identity}")

# Tính định thức (Determinant)
det = np.linalg.det(square_matrix)
print(f"\nĐịnh thức của ma trận: {det:.2f}")

# Ma trận nghịch đảo (nếu định thức != 0)
if det != 0:
    inverse = np.linalg.inv(square_matrix)

```

```

print(f"\nMa trận nghịch đảo:\n{inverse}")

# Kiểm tra:  $A * A^{-1} = I$ 
check = square_matrix @ inverse
print(f"\nKiểm tra  $A @ A^{-1} \approx I$ :\n{np.round(check, 2)}")

# Chuyển vị (đã học buổi sáng)
transpose = square_matrix.T
print(f"\nMa trận chuyển vị:\n{transpose}")

# Trace (tổng đường chéo chính)
trace = np.trace(square_matrix)
print(f"\nTrace (tổng đường chéo): {trace}")

```



TODO 1: Thực hành Matrix Calculation

Nhiệm vụ:

1. Tạo ma trận A kích thước 3x2 và ma trận B kích thước 2x4
2. Thực hiện phép nhân ma trận A @ B
3. Tạo ma trận vuông 4x4 với các số ngẫu nhiên từ 1-10
4. Tính định thức, ma trận nghịch đảo (nếu có), và trace
5. Kiểm tra tính chất $A @ A^{-1} = I$

```

In [ ]: # TODO 1: Viết code ở đây
# Bước 1: Tạo ma trận A (3x2) và B (2x4)

# Bước 2: Nhân ma trận A @ B

# Bước 3: Ma trận vuông 4x4 ngẫu nhiên

# Bước 4: Tính định thức, nghịch đảo, trace

# Bước 5: Kiểm tra  $A @ A^{-1} = I$ 

```

2. Array Manipulation - Thao tác Arrays nâng cao

2.1 Concatenate và Split Arrays

```

In [ ]: print("=== Concatenate (Nối arrays) ===")

# Tạo arrays để demo
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])
arr3 = np.array([[9, 10]])

print(f"Array 1:\n{arr1}")
print(f"Array 2:\n{arr2}")
print(f"Array 3:\n{arr3}")

# Nối theo axis=0 (theo hàng)

```

```

concat_vertical = np.concatenate([arr1, arr2], axis=0)
print(f"\nNối theo hàng (axis=0):\n{concat_vertical}")

# Nối theo axis=1 (theo cột)
concat_horizontal = np.concatenate([arr1, arr2], axis=1)
print(f"\nNối theo cột (axis=1):\n{concat_horizontal}")

# Sử dụng vstack và hstack (cách khác)
vstack_result = np.vstack([arr1, arr3]) # vertical stack
hstack_result = np.hstack([arr1, arr2]) # horizontal stack

print(f"\nVStack (arr1 + arr3):\n{vstack_result}")
print(f"\nHStack (arr1 + arr2):\n{hstack_result}")

print("\n=== Split (Tách arrays) ===")

# Tạo array lớn để tách
big_array = np.arange(12).reshape(4, 3)
print(f"Array lớn (4x3):\n{big_array}")

# Tách theo hàng
split_rows = np.split(big_array, 2, axis=0) # Tách thành 2 phần theo hàng
print(f"\nTách thành 2 phần theo hàng:")
for i, part in enumerate(split_rows):
    print(f"Phần {i+1}:\n{part}")

# Tách theo cột
split_cols = np.split(big_array, 3, axis=1) # Tách thành 3 phần theo cột
print(f"\nTách thành 3 phần theo cột:")
for i, part in enumerate(split_cols):
    print(f"Phần {i+1}:\n{part}")

```

2.2 Boolean Indexing và Advanced Indexing

```

In [ ]: print("=== Boolean Indexing ===")

# Tạo array điểm số
scores = np.array([85, 92, 78, 96, 65, 88, 73, 90, 82, 94])
print(f"Điểm số: {scores}")

# Tạo mask (mặt nạ boolean)
high_scores_mask = scores >= 85
print(f"\nMask (điểm >= 85): {high_scores_mask}")

# Lọc điểm cao
high_scores = scores[high_scores_mask]
# Hoặc viết ngắn gọn: high_scores = scores[scores >= 85]
print(f"Điểm cao (>= 85): {high_scores}")

# Các điều kiện khác
medium_scores = scores[(scores >= 70) & (scores < 85)]
low_scores = scores[scores < 70]

print(f"Điểm trung bình (70-84): {medium_scores}")
print(f"Điểm thấp (< 70): {low_scores}")

```

```

# Thay đổi giá trị thỏa mãn điều kiện
scores_copy = scores.copy()
scores_copy[scores_copy < 70] = 70 # Nâng điểm thấp lên 70
print(f"\nSau khi nâng điểm: {scores_copy}")

print("\n=== Advanced Indexing với 2D Array ===")

# Ma trận điểm 2D (học sinh x môn)
class_scores = np.array([[85, 90, 78, 92],
                        [88, 76, 95, 82],
                        [91, 89, 73, 96],
                        [65, 78, 88, 79]])

subjects = ['Toán', 'Lý', 'Hóa', 'Anh']
print(f"Bảng điểm lớp:\n{class_scores}")

# Tìm học sinh có ít nhất 1 môn < 80
students_need_help = np.any(class_scores < 80, axis=1)
print(f"\nHọc sinh cần hỗ trợ: {np.where(students_need_help)[0] + 1}")

# Tìm môn có điểm trung bình thấp nhất
subject_avg = np.mean(class_scores, axis=0)
worst_subject = np.argmin(subject_avg)
print(f"Môn khó nhất: {subjects[worst_subject]} (TB: {subject_avg[worst_subject]:.1}")

# Lọc tất cả điểm >= 90
excellent_scores = class_scores[class_scores >= 90]
print(f"\nTất cả điểm xuất sắc (>= 90): {excellent_scores}")

```

2.3 Fancy Indexing và np.where

```

In [ ]: print("=== Fancy Indexing ===")

# Tạo array dữ liệu
data = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
print(f"Dữ liệu gốc: {data}")

# Fancy indexing với list/array indices
indices = [0, 2, 4, 6, 8] # Lấy phần tử ở vị trí chẵn
selected = data[indices]
print(f"Phần tử vị trí chẵn: {selected}")

# Fancy indexing với 2D
matrix = np.arange(20).reshape(4, 5)
print(f"\nMa trận 4x5:\n{matrix}")

# Lấy hàng 0, 2 và cột 1, 3, 4
selected_rows = [0, 2]
selected_cols = [1, 3, 4]
subset = matrix[np.ix_(selected_rows, selected_cols)]
print(f"\nSubset (hàng 0,2 - cột 1,3,4):\n{subset}")

print("\n=== np.where() ===")

```

```
# np.where như if-else cho arrays
grades = np.array([85, 92, 78, 96, 65, 88, 73, 90])
print(f"Điểm gốc: {grades}")

# Chuyển đổi điểm thành chữ cái
letter_grades = np.where(grades >= 90, 'A',
                        np.where(grades >= 80, 'B',
                        np.where(grades >= 70, 'C', 'D')))
print(f"Điểm chữ: {letter_grades}")

# Tìm vị trí thỏa mãn điều kiện
high_grade_positions = np.where(grades >= 90)
print(f"Vị trí điểm cao (>= 90): {high_grade_positions[0]}")
print(f"Giá trị điểm cao: {grades[high_grade_positions]}")

# Ứng dụng: Thay thế giá trị
adjusted_grades = np.where(grades < 70, grades + 5, grades) # Cộng 5 điểm cho điểm
print(f"\nĐiểm sau điều chỉnh: {adjusted_grades}")
```



TODO 2: Thực hành Array Manipulation

Nhiệm vụ:

1. Tạo 3 arrays: điểm Toán, Lý, Hóa cho 5 học sinh
2. Nối chúng thành ma trận điểm hoàn chỉnh
3. Tách ma trận thành từng môn riêng biệt
4. Tìm học sinh có điểm Toán ≥ 8.0
5. Sử dụng np.where để phân loại học sinh: Giỏi (TB ≥ 8), Khá (6.5-8), Trung bình (< 6.5)

```
In [ ]: # TODO 2: Viết code ở đây
# Bước 1: Tạo 3 arrays điểm môn

# Bước 2: Nối thành ma trận hoàn chỉnh

# Bước 3: Tách ma trận thành từng môn

# Bước 4: Tìm học sinh điểm Toán  $\geq 8.0$ 

# Bước 5: Phân loại học sinh bằng np.where
```

3. Broadcasting - Phép toán với arrays khác kích thước

```
In [ ]: print("=== Broadcasting cơ bản ===")

# Array 2D và 1D
matrix = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

vector = np.array([10, 20, 30])
```

```

print(f"Ma trận 3x3:\n{matrix}")
print(f"Vector 1x3: {vector}")

# Broadcasting: matrix + vector
result = matrix + vector
print(f"\nMa trận + Vector (broadcasting):\n{result}")

print("\n=== Ứng dụng thực tế: Chuẩn hóa điểm ===")

# Bảng điểm thô
raw_scores = np.array([[85, 90, 78],
                        [92, 88, 76],
                        [79, 95, 82],
                        [88, 87, 90]])

subjects = ['Toán', 'Lý', 'Hóa']
print(f"Điểm thô:\n{raw_scores}")

# Tính điểm trung bình từng môn
subject_means = np.mean(raw_scores, axis=0)
print(f"\nĐiểm TB từng môn: {subject_means}")

# Chuẩn hóa: trừ đi trung bình (centering)
centered_scores = raw_scores - subject_means
print(f"\nĐiểm sau chuẩn hóa (trừ TB):\n{centered_scores}")

# Chuẩn hóa z-score: (x - mean) / std
subject_stds = np.std(raw_scores, axis=0)
z_scores = (raw_scores - subject_means) / subject_stds
print(f"\nZ-scores:\n{z_scores}")

print("\n=== Broadcasting với nhiều chiều ===")

# Ma trận 3D (ví dụ: điểm 3 Lớp x 4 học sinh x 3 môn)
class_data = np.random.randint(60, 100, (3, 4, 3))
print(f"Dữ liệu 3 lớp (3x4x3):")
print(f"Shape: {class_data.shape}")

# Cộng thêm điểm thưởng cho từng Lớp
class_bonus = np.array([2, 5, 3]).reshape(3, 1, 1) # Reshape để broadcast
adjusted_scores = class_data + class_bonus

print(f"\nĐiểm thưởng từng lớp: {class_bonus.flatten()}")
print(f"Điểm sau điều chỉnh có shape: {adjusted_scores.shape}")

```

4. Image as Arrays Intro - Giới thiệu ảnh như Arrays

4.1 Hiểu ảnh dưới dạng arrays

```

In [ ]: print("=== Ảnh là gì trong máy tính? ===")
        print("- Ảnh xám: Ma trận 2D (height x width)")
        print("- Ảnh màu: Ma trận 3D (height x width x channels)")
        print("- Mỗi pixel có giá trị 0-255 (hoặc 0-1)")

```

```

print("- 0 = đen, 255 = trắng")

# Tạo ảnh xám đơn giản bằng numpy
print("\n=== Tạo ảnh xám đơn giản ===")

# Ảnh xám 8x8 pixels
gray_image = np.array([
    [0, 0, 0, 0, 0, 0, 0, 0 ],
    [0, 50, 50, 50, 50, 50, 50, 0 ],
    [0, 50, 255, 255, 255, 255, 50, 0 ],
    [0, 50, 255, 0, 0, 255, 50, 0 ],
    [0, 50, 255, 0, 0, 255, 50, 0 ],
    [0, 50, 255, 255, 255, 255, 50, 0 ],
    [0, 50, 50, 50, 50, 50, 50, 0 ],
    [0, 0, 0, 0, 0, 0, 0, 0 ]
])

print(f"Ảnh xám 8x8:")
print(gray_image)
print(f"Shape: {gray_image.shape}")
print(f"Data type: {gray_image.dtype}")
print(f"Min value: {np.min(gray_image)}, Max value: {np.max(gray_image)}")

# Hiển thị ảnh dưới dạng ký tự
def display_image_as_text(img):
    """Hiển thị ảnh bằng ký tự ASCII"""
    for row in img:
        line = ""
        for pixel in row:
            if pixel == 0:
                line += "■" # Đen
            elif pixel < 100:
                line += "▀" # Xám đậm
            elif pixel < 200:
                line += "░" # Xám nhạt
            else:
                line += " " # Trắng
        print(line)

print("\nHiển thị ảnh bằng ký tự:")
display_image_as_text(gray_image)

```